Layer10 assignment

Aditya, 24B0919

March 2026

Contents

1 Introduction **1**

2 Ontology & Data extraction into a structured format **2**

3 Deduplication Strategy **2**

4 Updation **3**

5 Adaptation for Layer10 **3**
5.1 Fusion of Unstructured & Structured Data 3
5.2 Long-Term Memory & Preventing Drift 3
5.3 Grounding & Safety 4
5.3.1 Provenance 4
5.3.2 Deletions/Redactions 4
5.4 Permissions 4
5.5 Operational Reality 4

6 Querying **4**
6.1 query 1 4
6.1.1 result 1 5
6.2 query 2 5
6.2.1 result 2 6
6.3 Query 3 6
6.3.1 result 3 6

7 Visualization **7**

8 Running instructions **7**

1 Introduction

We used the ENRON email dataset, and mainly `llama-3.1-8b-instant` for inference. My code is in the `full_dev.ipynb` file, and is separated into cells to

demonstrate the complete pipeline—from data cleaning and LSH-based deduplication to long-term memory management—offering a more modular and explorable interface than a standard static script.

2 Ontology & Data extraction into a structured format

The ontology was designed to capture professional relationships, project involvement, and administrative actions within an organizational context. It was made of two primary components: Entities and Predicates. The claims were structured in the "subject Predicate object" manner, where subject and object are entities. I chose a combinatation of predicate and entity allowed values which led to the best combination of hallucinations and accurately representing the data and so was limited by model accuracy.

We used allowed entity types:

```
["Person", "Organization", "Location", "Project", "
    Award", "Asset", "Event", "Job", "Meeting", "
    Document", "Concept"]
```

and allowed predicates:

```
["WORKS_ON", "WORKS_UNDER", "CAUSES_ISSUE_IN", "
    RESOLVED", "NOT_RESOLVED", "MANAGES", "LOCATED_IN
    ", "WON", "OWNS_OR_CONTROLS", "ATTENDED", "
    AUTHORED", "INVITED", "WAS_COMPLETED_BY", "
    DISCUSSING", "OFFERS", "OTHER"]
```

3 Deduplication Strategy

We followed distinct strategies for deduplication:

- **Artifact deduplication** was done before structured extraction and included cleaning up of text into small indivisible parts and then checking for exact and partial similarity. If similarity was above a certain threshold, we didn't do inference on that text again, but added it as a part of evidence to the claims made using the other similar text.

- **Claim deduplication** was done while doing structured extraction itself. Since claims were short and of the form 'subject predicate object', equality of claims could be easily checked, duplicate claims weren't created but the new text was added as evidence.

- **Entity deduplication** was done later asynchronously. First we make the graphical dataset using the claims extracted above, then we extracted a list of items that could potentially be referring to the same thing, used

LLM for inferring if that was actually the case, and if it was, then we made the two nodes point to a single canonical node.

4 Updation

For updation, by default, we check all nodes where there is a mismatch between prompt version, ontology version, or model used, we use the files used in inferring those nodes, remove all the other nodes inferred using those files, and resend those files to the LLM for inference. We also allow a custom query input to our function, which will be the more commonly used case, because often we would want only a certain category of nodes to get inferred, using say a new LLM model, and not the whole dataset. Also when files are changed, we can call the same function with a custom query which selects nodes which mention our file (for evidence), this would lead to deletion of those nodes and then new nodes will be inferred and inserted into the graph.

5 Adaptation for Layer10

I would do the following to adapt my current model to Layer10's work.

5.1 Fusion of Unstructured & Structured Data

The current ontology focuses on high-level entities like Person, Project, and Concept. I would expand the extraction contract to include "Linking Entities": Add Ticket (Jira/Linear), Channel (Slack/Teams), and PR (GitHub) as core entities, and for example, use `CAUSES_ISSUE_IN` and `RESOLVED` predicates to link a Slack discussion directly to a Jira Ticket ID, also some more contextual mapping.

5.2 Long-Term Memory & Preventing Drift

We already have a `decay` function but its more basic. For Layer10's needs we could have seperate decay rates for durable and more-transient kind of claims. For example day-to-day chat banter or transient status updates could be assigned a much higher `reduction_factor` in the decay function.

For drift prevention, when a new claim contradicts an old PERMANENT one the system could use the `last_updated_time` logic to trigger a "Truth Re-evaluation", also the newer evidence should reduces the confidence of the conflicting legacy claim. (Currently we have a function which does something similar: `get_and_fix_unique` but instead of reducing confidence it deletes older conflicting nodes if so commanded).

5.3 Grounding & Safety

5.3.1 Provenance

My system already tracks `file_id`, `quote`, and `time_stamp`. For Layer10, this expands to `source_url` and `author_id`.

5.3.2 Deletions/Redactions

I already have a file footprint cleaning function `wipe_file_footprint`. If a Slack message is deleted, the system executes this function to remove the orphaned Evidence nodes and potentially delete the Claim if no other source supports it.

5.4 Permissions

I will make a function such that when a user queries, permissions are appropriately set, and the graph on which search is carried out is the subgraph on which none of the nodes use those files as evidence which are not permitted to be seen by the user.

5.5 Operational Reality

We already have the incremental updates logic in our `extract_claims_partial` function. That logic could be triggered by webhooks, for example a Jira update or a new email, to process only the change.

We try to use less resources by comparing similarity of a new message to previous messages, and not calling the LLM API on repeated content. This could be useful to Layer10.

For regression testing, we can have a baseline dataset of claims and confidence. Any change to the `prompt_version` is tested against this baseline, and predicate and confidence changes are found out. If they differ significantly then we conclude there is something wrong with the new model/prompt.

6 Querying

We attach the results obtained by a few queries on the database.

6.1 query 1

```
MATCH (canon:CanonicalEntity)
WITH canon, rand() AS r
ORDER BY r
LIMIT 1
MATCH (alias:Entity)-[:RESOLVES_TO]->(canon)
```

```
OPTIONAL MATCH (alias)-[:SUBJECT_OF]->(outClaim:Claim {
    status: 'PERMANENT'})-[:HAS_OBJECT]->(target:Entity)
OPTIONAL MATCH (source:Entity)-[:SUBJECT_OF]->(inClaim:
    Claim {status: 'PERMANENT'})-[:HAS_OBJECT]->(alias)

RETURN
    canon.name AS canonical_name,
    canon.type AS type,
    [rel IN collect(DISTINCT {
        predicate: outClaim.predicate,
        to: target.name,
        confidence: outClaim.confidence
    }) WHERE rel.predicate IS NOT NULL] AS
       outgoing_claims,
    [rel IN collect(DISTINCT {
        predicate: inClaim.predicate,
        from: source.name,
        confidence: inClaim.confidence
    }) WHERE rel.predicate IS NOT NULL] AS
       incoming_claims
```

6.1.1 result 1


```
{'canonical_name': 'brian redmond', 'type': 'Person', '
    outgoing_claims': [{'to': 'daily lunches', 'predicate
    ': 'ATTENDED', 'confidence': 0.9}, {'to': 'enron', '
    predicate': 'LOCATED_IN', 'confidence': 0.9}, {'to': '
    enron', 'predicate': 'MANAGES', 'confidence': 0.9}, {'
    to': 'mark russ', 'predicate': 'MANAGES', 'confidence
    ': 0.9}, {'to': 'producer/wellhead group (texas)', '
    predicate': 'MANAGES', 'confidence': 0.9}], '
    incoming_claims': [{'predicate': 'INVITED', '
    confidence': 0.9, 'from': 'louise kitchen'}, {'
    predicate': 'INVITED', 'confidence': 1.0, 'from': '
    louise'}]}
```


Note that we have not finally connected to a canonical entity in this query.

6.2 query 2

```
MATCH (sub:Entity)-[:SUBJECT_OF]->(c:Claim {status: '
    NEEDS_REVIEW'})-[:HAS_OBJECT]->(obj:Entity)
WHERE c.predicate IS NOT NULL
  AND sub.name IS NOT NULL
  AND obj.name IS NOT NULL
```

```
MATCH (c)-[:SUPPORTED_BY]->(ev:Evidence)
WHERE ev.quote IS NOT NULL
WITH sub, c, obj, count(ev) AS evidence_count

WHERE evidence_count > 2 AND c.confidence < 0.7

RETURN
    sub.name AS subject,
    c.predicate AS predicate,
    obj.name AS object,
    round(c.confidence, 2) AS current_confidence,
    evidence_count AS total_sources
ORDER BY evidence_count DESC
```

6.2.1 result 2

`Null`. This is justified, because most claims which have high confidence are already set to "PERMANENT" and high evidence count correlates with high confidence.

6.3 Query 3

```
MATCH (e:Entity)-[:SUBJECT_OF]->(c:Claim {status: '
    PERMANENT'})
WHERE e.name IS NOT NULL
  AND c.predicate IS NOT NULL

WITH e, count(DISTINCT c) AS claim_count, collect(
    DISTINCT c.predicate) AS activities
WHERE claim_count >= 3
RETURN
    e.name AS entity_name,
    e.type AS type,
    claim_count AS total_permanent_claims,
    activities AS types_of_relations
ORDER BY total_permanent_claims DESC
LIMIT 10
```

6.3.1 result 3

```
{'entity_name': 'enron', 'type': 'Organization', '
    total_permanent_claims': 76, 'types_of_relations': ['
    OFFERS', 'LOCATED_IN', 'MANAGES', 'OWNS_OR_CONTROLS',
    'WON', 'CAUSES_ISSUE_IN']}
```

7 Visualization

We attach a few screenshots of the graph structure on various queries.

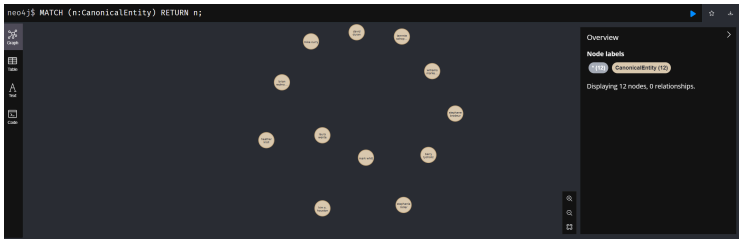

Figure 1: All canonical nodes

Figure 2: Edges connected to canonical nodes

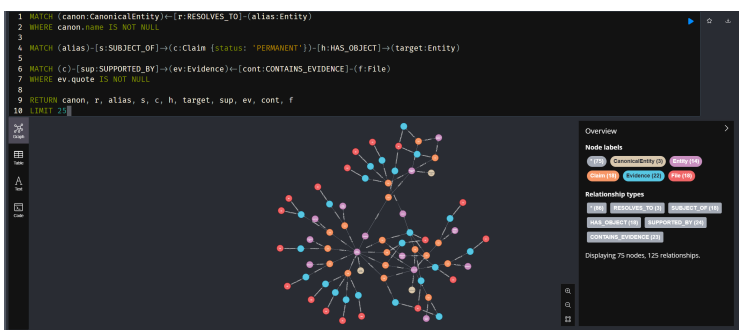

Figure 3: Full matching from canonical nodes to evidence

8 Running instructions

- Download the enron dataset from `https://www.cs.cmu.edu/~enron/enron_mail_20150507.tar.gz`

- untar the file downloaded above and place the resulting `maildir` folder in the same directory as the `full_dev.ipynb` file from my repo.
- Make a neo4j database, and API keys, and substitute them in the first cell of `Data extraction and structuring using LLM` section.
- Execute all the cells in the notebook till the cell with a call to `process_extracted_claims`. At this point the graph should be made, and can be visualized at port 7474 in browser. Note that nodes that need review might be garbage.
- The functionality of each function is mentioned in comments or docstrings. They can be used as per the appropriate function signature. (I have not used many functions myself in the notebook)
- Executing all the cells before the `Extra work for submission` section will ensure proper deduplication, the `lsh_index` and `hash_vals` (hash values of seen files) will be saved in a pickle file.
- Some other commonly used functions might be
  - `decay`: reduces confidence of older memory and marks for human review if needed.
  - `extract_claims_partial`: changes to sub-parts of graph database with back-filling.
  - `undo_entity_merge`: If after querying the database, we feel some canonical entities are mapped to entities which are not same physically, we can use this function to remove the relation between them.